

AUTOMATION • API & WEBHOOK

# Автоматизация обработки входящей заявки

Архитектура интеграции, контракт webhook, алгоритм и контроль  
надёжности

REST API • webhook • FastAPI • idempotency • observability

Автор портфолио: Леонид (ExpertStudy)

31 августа 2026 • Все данные и сценарии вымышлены

ДЕМОНСТРАЦИОННАЯ УЧЕБНАЯ РАБОТА • НЕ КЛИЕНТСКИЙ ЗАКАЗ

## Содержание и паспорт работы

Краткая навигация по демонстрационному материалу.

- 01 Исходный процесс и цель
- 02 Архитектура решения
- 03 Контракт webhook
- 04 Обработчик и оркестрация
- 05 Безопасность, тесты и метрики

### Паспорт работы

|              |                                                          |
|--------------|----------------------------------------------------------|
| Формат       | Техническая аналитическая записка с архитектурной схемой |
| Сценарий     | Передача заявки сайта в CRM, задачи и уведомления        |
| Стек примера | REST/JSON, FastAPI, очередь, PostgreSQL/Redis            |
| Результат    | Контракт, код, правила повторов, тест-план и KPI         |

**Статус** Материал создан как самостоятельный учебный пример для портфолио. Он показывает подход, стиль объяснения и уровень оформления; реальные заказчики и их данные не использовались.

## 01 Исходный процесс и цель

Ручной перенос заявки заменяется наблюдаемым и повторяемым процессом.

В учебном сценарии пользователь заполняет форму на сайте. Оператор копирует контакт в CRM, создаёт задачу менеджеру и отправляет уведомление в рабочий чат. Ручной процесс занимает 8-15 минут и иногда приводит к пропускам или дублированию.

### Проблемы текущего состояния

- данные копируются между системами и могут искажаться;
- нет единого идентификатора события и защиты от дублей;
- ошибка одного внешнего сервиса блокирует весь процесс;
- время обработки и причина сбоя не измеряются.

### Целевые показатели

| Показатель          | Цель         | Способ контроля            |
|---------------------|--------------|----------------------------|
| Приём webhook       | p95 < 800 мс | Метрика HTTP latency       |
| Создание лида в CRM | 99% < 2 мин  | event_id + timestamp       |
| Дубли               | < 0,1%       | Уникальный idempotency key |
| Потерянные события  | 0            | Очередь + DLQ + сверка     |

**Принцип** Webhook должен быстро подтвердить приём события. Долгие вызовы CRM и уведомлений выполняются асинхронно.

02 Архитектура решения

Компоненты разделяют приём, бизнес-логику и внешние интеграции.

Архитектура обработки webhook

Синхронный приём - асинхронное выполнение - наблюдаемость

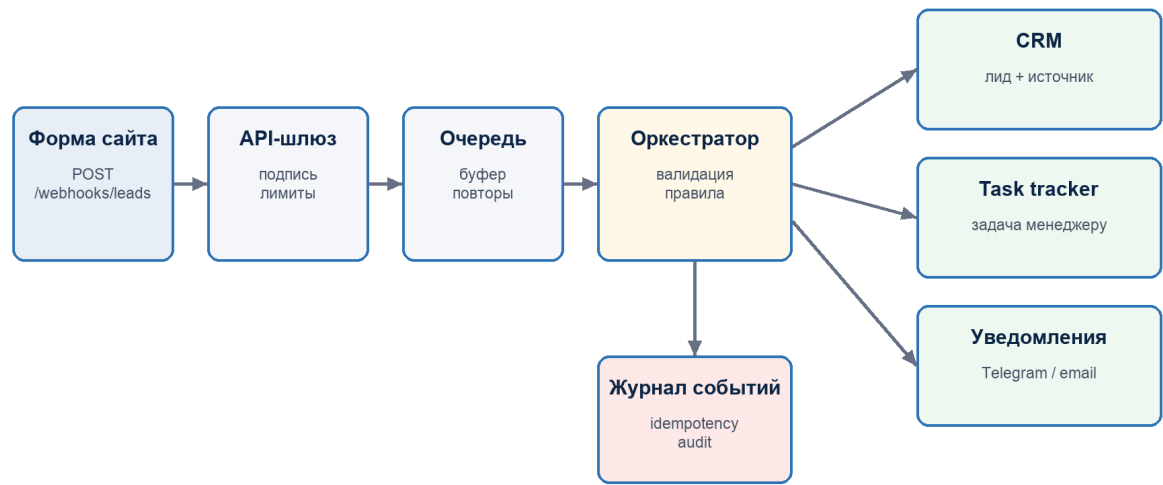


Рисунок 1. Предлагаемая архитектура автоматизации

| Компонент       | Ответственность                     | Отказоустойчивость                    |
|-----------------|-------------------------------------|---------------------------------------|
| API-шлюз        | TLS, лимиты, проверка подписи       | Отклоняет неверные запросы до очереди |
| Webhook handler | Валидация схемы, event_id, HTTP 202 | Не ждёт CRM и уведомления             |
| Очередь         | Буферизация события                 | Повторы и DLQ после лимита            |
| Оркестратор     | Маппинг, правила маршрутизации      | Шаги повторяются независимо           |
| Журнал          | Статусы и correlation_id            | Позволяет сверять и переигрывать      |

**Граница транзакции** Внутри сервиса атомарно фиксируются event\_id и факт постановки в очередь. Внешние системы согласуются по модели eventual consistency.

## 03 Контракт webhook

Явная схема входного события уменьшает число неявных преобразований.

### Листинг 1. Демонстрационный JSON payload

```
{
  "event_id": "evt_20260831_8f13c9",
  "event_type": "lead.created",
  "occurred_at": "2026-08-31T09:42:17Z",
  "source": "landing_page",
  "lead": {
    "name": "Демо Пользователь",
    "phone": "+7XXXXXXXXXX",
    "email": "demo@example.test",
    "topic": "Автоматизация отчётности",
    "consent": true
  },
  "utm": {
    "source": "search",
    "campaign": "demo_campaign"
  }
}
```

| Поле             | Обязательное | Правило                                                    |
|------------------|--------------|------------------------------------------------------------|
| event_id         | Да           | Уникальный ключ; повтор возвращает уже известный результат |
| event_type       | Да           | Версионизируемое значение lead.created                     |
| occurred_at      | Да           | ISO 8601 в UTC                                             |
| lead.phone/email | Одно из двух | Нормализация перед записью в CRM                           |
| lead.consent     | Да           | Без true событие не передаётся в маркетинговый контур      |
| utm              | Нет          | Сохраняется без влияния на основной маршрут                |

### Версионирование

Критичные несовместимые изменения публикуются как новый тип или новая версия endpoint, например /v2/webhooks/leads. Добавление необязательного поля обратно совместимо.

## 04 Обработчик и оркестрация

Пример показывает проверку HMAC-подписи и идиempотентный приём.

### Листинг 2. Сокращённый обработчик FastAPI

```
import hashlib
import hmac
from fastapi import FastAPI, Header, HTTPException, Request, status

app = FastAPI()
WEBHOOK_SECRET = b"read-from-secret-manager"

def valid_signature(body: bytes, provided: str) -> bool:
    expected = hmac.new(WEBHOOK_SECRET, body, hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, provided)

@app.post("/webhooks/leads", status_code=status.HTTP_202_ACCEPTED)
async def accept_lead(request: Request, x_signature: str = Header(...)):
    body = await request.body()
    if not valid_signature(body, x_signature):
        raise HTTPException(status_code=401, detail="invalid signature")

    event = await request.json()
    event_id = event["event_id"]
    created = await event_store.save_if_absent_and_enqueue(event_id, event)
    if not created:
        return {"status": "already_accepted", "event_id": event_id}
    return {"status": "accepted", "event_id": event_id}
```

### Алгоритм фонового обработчика

1. получить событие из очереди и установить `correlation_id = event_id`;
2. проверить бизнес-правила и привести телефон/email к каноническому виду;
3. создать или обновить лид в CRM по внешнему идентификатору;
4. создать задачу ответственному с дедлайном по SLA;
5. отправить уведомление и записать статусы каждого шага;
6. подтвердить сообщение только после фиксации итогового состояния.

**Важно** `save_if_absent_and_enqueue` - условная атомарная операция. В промышленной реализации её обеспечивает уникальный ключ `event_id` вместе с `outbox`-паттерном либо транзакционной очередью.

## 05 Повторы и обработка ошибок

Временная недоступность внешней системы не должна создавать дубли.

| Ситуация            | Действие                                         | Результат                    |
|---------------------|--------------------------------------------------|------------------------------|
| CRM вернула 429     | Повтор с экспоненциальной задержкой и jitter     | Снижается нагрузка на API    |
| CRM вернула 400     | Не повторять; записать validation_error          | Событие требует исправления  |
| Таймаут             | Проверить статус по external_id, затем повторить | Исключается двойное создание |
| 5 неудачных попыток | Переместить в DLQ и оповестить                   | Событие не теряется          |
| Повторный webhook   | Вернуть сохранённый статус по event_id           | Идемпотентный ответ          |

### Листинг 3. Классификация ошибок

```

delay_seconds = min(300, 2 ** attempt) + random.uniform(0, 1)

if response.status_code in {429, 500, 502, 503, 504}:
    schedule_retry(event_id, delay_seconds)
elif 400 <= response.status_code < 500:
    mark_failed(event_id, reason="non_retryable")
else:
    mark_step_completed(event_id, step="crm")

```

### Наблюдаемость

- структурированные логи: event\_id, step, attempt, duration\_ms, result;
- метрики: accepted\_total, failed\_total, retry\_total, queue\_age\_seconds;
- трассировка: единый correlation\_id во всех исходящих запросах;
- алерт: возраст старейшего сообщения выше 120 секунд или рост DLQ.

## 06 Безопасность, тесты и выводы

Качество интеграции проверяется до подключения реальных систем.

### Минимальные меры безопасности

- HTTPS, HMAC-подпись тела запроса и ротация секрета;
- защита от replay: timestamp + ограниченное окно приёма;
- секреты только в secret manager, без записи payload с персональными данными в открытые логи;
- минимальные права сервисного аккаунта CRM и ограничение частоты запросов;
- срок хранения событий и процедура удаления данных.

### Тест-план

| Тест                    | Ожидаемый результат                                  |
|-------------------------|------------------------------------------------------|
| Корректный payload      | HTTP 202, один лид и одна задача                     |
| Неверная подпись        | HTTP 401, событие не сохраняется                     |
| Повтор event_id         | Новых объектов нет; возвращается прежний статус      |
| CRM недоступна 3 минуты | Повторы; событие обрабатывается после восстановления |
| Постоянная ошибка 400   | Статус failed, запись причины, без повторов          |
| Нагрузочный тест        | p95 приёма ниже 800 мс при заданном RPS              |

### Итог

Предложенная схема сокращает ручные операции, сохраняет входящие события и позволяет контролировать каждый шаг. Ключевые свойства - быстрый приём webhook, идемпотентность, повторяемость операций и измеримый SLA.

**Результат примера** Продемонстрированы навыки описания бизнес-процесса, проектирования API-интеграции, работы с webhook, обработки ошибок, безопасности и технической документации.